
SOUTHEAST UNIVERSITY

MediCare Hospital Management System

Patient Management Module — Design & Implementation Report

Submitted By

Name	Student ID
MD RAZWAN AHMMAD	2024200000396
ABDULLAH AL NOMAN	2024200000361
MD YOUSUF ALI	2024000000366

Submitted To

Md. Ayon Mia

Lecturer, Southeast University

Project Introduction & Objective

The MediCare Hospital Management System is a Java desktop application meant to take basic hospital record-keeping off paper and out of scattered notebooks. Instead of separate registers for patients, doctors, appointments and bills, everything sits in one program that a receptionist or admin can actually search through in a few seconds.

For this stage of the project, the Patient module has been carried through to working code: a patient class hierarchy built around inheritance and polymorphism, and a separate record-management class that handles adding, viewing, updating and deleting patient data from a text file, with proper validation and a custom exception. The rest of the system (doctors, appointments, billing) follows the same pattern once this module is wired into the Swing interface.

Add & Store Patients

New patient records are validated, then written to a text file so nothing is lost on exit.

View All Records

Every saved record can be listed straight from the file, one line per patient.

Update a Record

An existing patient's name, phone or disease can be corrected by ID without touching other rows.

Delete a Record

A patient record can be removed by ID, and the file is rewritten without that line.

Problem Statement

Hospitals that still track patients on paper or in loose spreadsheets run into the same handful of problems again and again:

Scattered Records

Patient details end up split across registers, sticky notes and separate files.

Slow Searching

Finding one patient's history means flipping through pages by hand.

Data Entry Mistakes

Nothing stops a blank name or phone number from being written down.

No Safe Update Path

Editing a paper record usually means crossing it out and rewriting it.

Solution: The Patient module stores every record in a structured text file through a dedicated manager class, checks required fields before saving, and raises a custom exception instead of letting bad data slip through.

OOP Concept Implementation

The code demonstrates all four pillars of Object-Oriented Programming, each tied to an actual class rather than just described in theory:

Encapsulation

Patient, Person and PatientException keep their fields private and expose them only through getters, setters or a constructor.

Abstraction

The abstract Person class defines name, phone and displayInfo() without saying how a specific person type should print its details.

Inheritance

Patient and Doctor both extend Person, reusing name and phone while adding patientId/disease or specialization of their own.

Polymorphism

Calling displayInfo() on a Person reference runs the Patient or Doctor version depending on the actual object, shown directly in Main.

Class Structure & Design

The module is really two small demonstrations sharing one idea — a Patient. The first shows the class hierarchy (Person, Patient, Doctor); the second is a separate, simpler Patient record used by the file-based CRUD manager.

Class	Key Variables / Fields	Key Methods / Functions
Person (<i>Abstract</i>)	name, phone	displayInfo() (<i>abstract</i>), getName(), getPhone()
Patient extends Person	patientId, disease	displayInfo()
Doctor extends Person	specialization	displayInfo()
Patient (<i>record class</i>)	id, name, phone, disease	toString(), getName(), getPhone(), getDisease(), setName(), setPhone(), setDisease()
PatientManager	fileName	addPatient(), viewPatients(), updatePatient(), deletePatient()
PatientException extends Exception	message (<i>inherited</i>)	standard exception constructor

Abstract Base Class

Person cannot be instantiated directly — the code always creates a Patient or a Doctor.

Inheritance Chain

Patient and Doctor both extend Person, so they inherit name and phone automatically.

File-Based Storage

PatientManager reads and writes every patient record as a comma-separated line in patients.txt.

Key Methods & Functions

Patient Record Management (*PatientManager class*)

- **addPatient(Patient p)** — throws PatientException if name or phone is empty, otherwise appends the record to patients.txt
- **viewPatients()** — reads every line from patients.txt and prints it to the console
- **updatePatient(id, newName, newPhone, newDisease)** — rewrites the file with the matching ID's line replaced
- **deletePatient(id)** — rewrites the file leaving out the line with the matching ID

OOP Demonstration (*Person / Patient / Doctor*)

- **displayInfo()** — overridden separately in Patient and Doctor to print type-specific details
- Main creates a Patient and a Doctor, stores both in Person variables, and calls displayInfo() on each to show the correct version runs at execution time

Data, Variables & I/O

Core Variables

- Person: String name, phone
- Patient (subclass): int patientId, String disease
- Doctor: String specialization
- Patient (record): int id, String name, phone, disease

Inputs

- Patient/Doctor test data passed straight into the constructors in Main
- New id, name, phone, disease values passed into addPatient() / updatePatient()
- An id value passed into updatePatient() / deletePatient() to locate a record

Outputs

- Console printout of patient/doctor details through displayInfo()
- Comma-separated lines written to patients.txt (id, name, phone, disease)
- Status messages on the console, e.g. "Patient added successfully.", "Patient updated.", "Patient deleted."

Exception Handling Strategy

Required fields are checked before anything is written to disk, and every file operation sits inside a try-catch block so a missing or unreadable file prints a message instead of stopping the program.

PatientException

A custom checked exception thrown by addPatient() when the name or phone field is left empty.

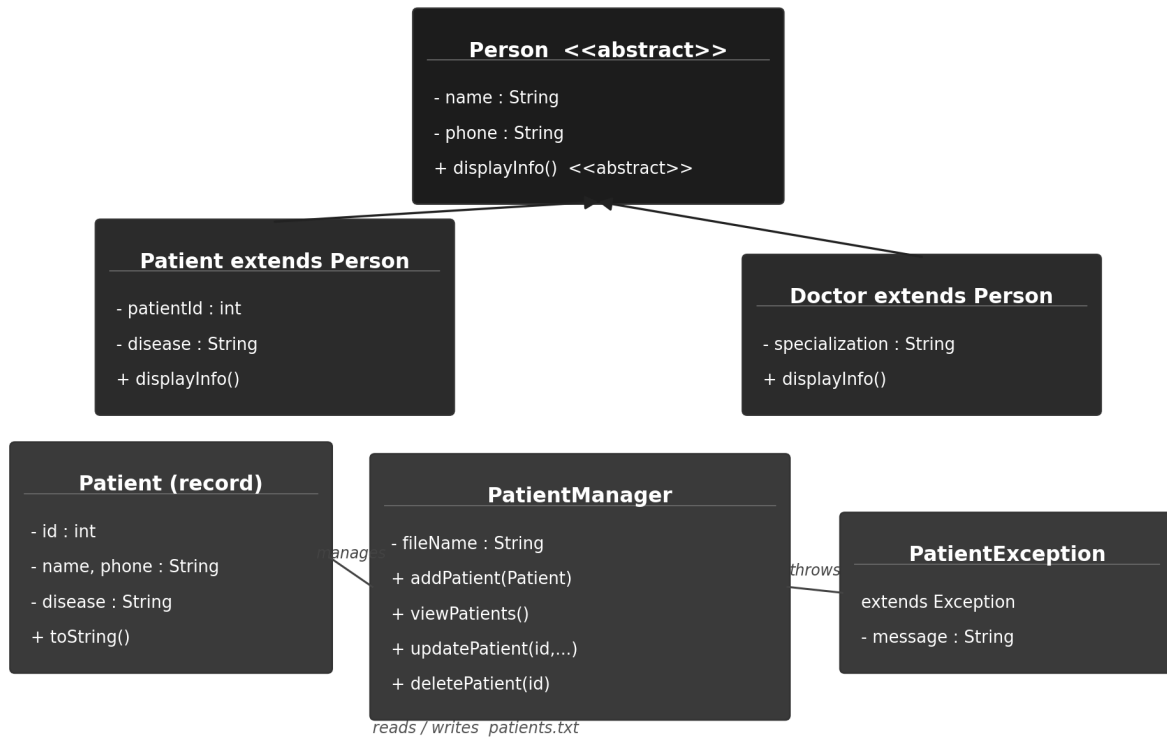
IOException Handling

FileWriter / FileReader / BufferedReader calls in PatientManager are wrapped in try-catch so read or write failures don't crash the program.

Implementation: Main calls addPatient() inside its own try-catch and prints the PatientException message if one is thrown, while PatientManager's file methods catch IOException internally and print a plain-language failure message instead of a stack trace.

Class Diagram

A simplified view of how the classes relate — the inheritance side at the top, the file-based CRUD side below it.



Current Scope & Next Step

The code written so far runs as a console program (two small demos: one for the Person/Patient/Doctor hierarchy, one for the file-based CRUD manager) rather than through the Swing interface described in the original proposal. That was a deliberate choice — it let the OOP design, file handling and exception handling get tested and fixed on their own before wiring them into a GUI.

The next step is to drop PatientManager and the Patient/Doctor classes behind a JTabbedPane with add, search, update and delete panels, exactly as laid out in the full project proposal, and then repeat the same pattern for Doctor, Appointment, MedicalRecord and Bill.

Conclusion

The Patient module covers the core ideas the course asks for: an abstract base class with two real subclasses, method overriding used through a superclass reference, a custom exception, and file-based CRUD operations wrapped in proper exception handling. It is a small piece of the full MediCare system, but it is a working one, and the same structure can be reused for the remaining modules without starting from scratch.